

Progetto di Apprendimento Automatico e Profondo

Object Detection e Sentiment Analysis: da YOLO ai Transformer

Matteo Barbieri

Apprendimento Automatico e Profondo

16 luglio 2026

- 1 Introduzione: reti neurali
- 2 Object Detection (YOLO)
- 3 Sentiment Analysis (Transformer)
- 4 Conclusioni

- 1 Introduzione: reti neurali
- 2 Object Detection (YOLO)
- 3 Sentiment Analysis (Transformer)
- 4 Conclusioni

Modello del neurone

Il neurone calcola una **combinazione lineare** degli input e vi applica una **funzione di attivazione** g :

$$z = \sum_{j=1}^n w_j x_j + b, \quad a = g(z)$$

- $w_j = \mathbf{pesi}$ (quanto conta ogni input), $b = \mathbf{bias}$.
- Con il trucco della *bias unit* $x_0 = 1$ si scrive compattamente $z = \theta^\top \mathbf{x}$.
- Senza g il neurone è solo un **prodotto scalare**: è g a dare la **non linearità**.

Dalla rete al forward propagation

I neuroni si organizzano in **layer**: l'output di un layer è l'input del successivo (*forward propagation*):

$$\mathbf{a}^{(l)} = g\left(W^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}\right)$$

Feature apprese

La rete **scopre da sola** le feature nascoste: la semantica la diamo noi. (Es. prezzo di una casa: un'unità nascosta può rappresentare la “dimensione della famiglia”).

Perché non lineari?

Con sole attivazioni **lineari** una rete profonda equivale a **un solo neurone**. Con attivazioni non lineari diventa un **approssimatore universale di funzioni**.

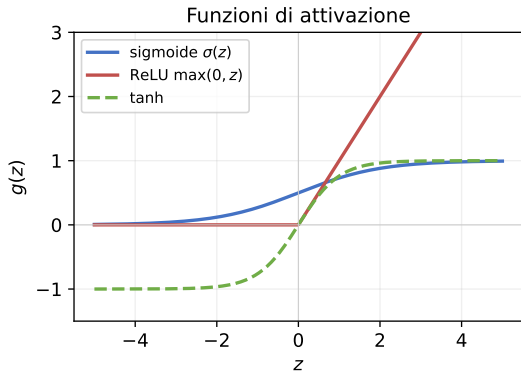
Funzioni di attivazione

$$\sigma(z) = \frac{1}{1 + e^{-z}} \quad \text{ReLU}(z) = \max(0, z)$$

- **Layer nascosti: ReLU** (oggi lo standard).
- **Output regressione:** lineare / identità.
- **Output binario:** sigmoide (logistica).
- **Output multiclasse: softmax.**

Il passaggio da *sigmoide* a *ReLU* è uno dei progressi algoritmici che hanno reso possibile il deep learning.

$$\text{softmax}(\mathbf{z})_k = \frac{e^{z_k}}{\sum_j e^{z_j}} \quad (\text{la logistica ne è il caso binario})$$



La funzione di costo

La rete produce $\hat{y}$; la **loss** misura quanto sbaglia su un esempio, il **costo** J è la media su tutti gli m esempi:

$$J(\theta) = \frac{1}{m} \sum_{i=1}^m L(\hat{y}^{(i)}, y^{(i)})$$

Regressione: errore quadratico

$$L = \frac{1}{2} (\hat{y} - y)^2$$

Il fattore $\frac{1}{2}$ semplifica la derivata.

Classificazione: cross-entropy

$$L = - \sum_k y_k \log \hat{y}_k$$

Nel progetto usiamo **entrambe**: errore quadratico per le coordinate delle bounding box, cross-entropy per le classi e il sentiment.

Apprendimento: gradient descent e backpropagation

Regola di aggiornamento

Si aggiornano i parametri nella direzione **opposta al gradiente**:

$$\theta := \theta - \alpha \frac{\partial J}{\partial \theta}$$

con $\alpha = \textbf{learning rate}$ (ampiezza del passo).

- Il gradiente si calcola con la **backpropagation**: si propaga l'errore all'indietro applicando la *regola della catena*, usando la derivata della funzione di attivazione di ogni unità.
- α troppo grande $\rightarrow$ si diverge; troppo piccolo $\rightarrow$ convergenza lentissima.

Ottimizzazione: dal mini-batch ad Adam

- **Mini-batch gradient descent:** aggiornare i pesi usando tutto il dataset è proibitivo $\Rightarrow$ si procede a blocchi (tipicamente 32–256 esempi). *Nel progetto: batch da 16.*

Migliorare la discesa (medie mobili esponenziali)

- **Momentum:** media mobile del *gradiente* $\rightarrow$ aiuta a uscire dai minimi locali e smorza le oscillazioni.
- **RMSprop:** media mobile del *quadrato* del gradiente $\rightarrow$ normalizza le fluttuazioni dividendo per la sua radice.
- **Adam:** combina Momentum e RMSprop, con *bias correction*.

Adam è l'ottimizzatore usato in **entrambi** gli esercizi.

Batch Normalization

Idea

Come si normalizzano gli input, si possono normalizzare anche i valori dei **nodì interni**, usando media e varianza calcolate su ogni **mini-batch** (poi riscalati con parametri apprendibili).

- Contrasta il **covariate shift**: i layer profondi ricevono input la cui distribuzione cambia di continuo durante il training.
- In pratica: si aggiunge un **layer BN** dopo il layer da normalizzare e si fa la solita backpropagation.
- A test time si usano le medie mobili accumulate durante il training.

Usata in ogni blocco convoluzionale del detector: $\text{Conv} \rightarrow \text{BN} \rightarrow \text{ReLU}$.

Dal neurone al deep learning

Idea del deep learning

Comporre (*nesting*) funzioni semplici in una sequenza di layer sempre più **astratti**: le *feature* vengono apprese insieme all'obiettivo (*representational learning*).

Approcci principali del corso: feedforward, **CNN** (immagini), reti ricorrenti, **Transformer** (sequenze), autoencoder, GAN.

I due esercizi usano due di questi approcci

- **Object detection** → rete **convoluzionale** (CNN).
- **Sentiment analysis** → **Transformer**.

- 1 Introduzione: reti neurali
- 2 Object Detection (YOLO)**
- 3 Sentiment Analysis (Transformer)
- 4 Conclusioni

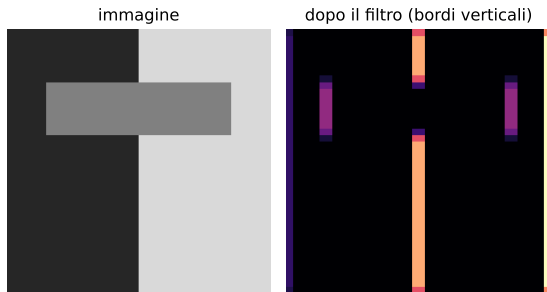
Reti convoluzionali: la convoluzione

Una rete densa su immagini grandi è **ingestibile** (troppi pesi).

La **convoluzione** fa scorrere un piccolo **filtro** (kernel) sull'immagine, calcolando un prodotto scalare a ogni posizione → produce una *feature map*.

Filtri appresi, non progettati

Un filtro può rilevare i **bordi** (verticali, orizzontali). Invece di costruirlo a mano (Sobel, Scharr), si **apprendono i pesi del filtro** con la backpropagation.



Un filtro verticale evidenzia i bordi verticali.

CNN: padding, stride, pooling

Iperparametri della convoluzione

- **Padding:** bordo di zeri; *same* mantiene la dimensione, *valid* la riduce.
- **Stride:** di quanti passi scorre il filtro.
- **Più filtri** → più feature map (profondità).
Su immagini RGB: convoluzione su ogni canale, poi somma.

Pooling

Il **max pooling** tiene il valore massimo di ogni regione: riduce la dimensione e agisce da rilevatore di feature.

Architettura tipica:

CONV → POOL → ... → FC

Andando in profondità: più filtri, dimensioni spaziali più piccole.

Perché la convoluzione funziona

Due proprietà chiave

- **Parameter sharing:** un rilevatore di feature (es. un bordo) è utile in **tutta** l'immagine → pochissimi parametri, che *non dipendono* dalla dimensione dell'immagine.
- **Sparsità delle connessioni:** ogni output dipende solo da una **piccola regione** dell'input → meno parametri, meno overfitting.

Nel detector: 5 blocchi Conv → BN → ReLU + max-pool riducono 224×224 fino alla griglia 7×7 .

Il problema: localizzazione con bounding box

Object detection

Data un'immagine, individuare **quali** oggetti contiene e **dove** si trovano, disegnando una *bounding box* attorno a ciascuno.

Dataset: Object365 (365 classi, 600k immagini, 10M box).

- Volume incompatibile con un portatile $\Rightarrow$ **subsampling** su due assi.
- **Classi**: da 365 a **8** frequenti (Person, Car, Chair, Bottle, Cup, Lamp, Hat, Book).
- **Immagini**: 4.867 tenute (50.071 box); solo quelle con almeno un oggetto utile.

Tecnica scelta: **YOLO**. Griglia, anchor, loss, IoU e NMS sono *implementati a mano* (nessuna libreria di detection).

Dalla sliding window a YOLO

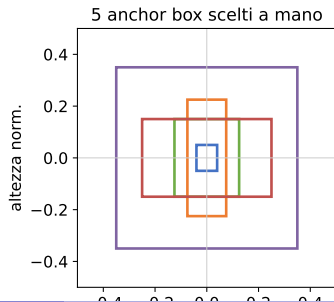
Sliding window: si fa scorrere una finestra e si classifica ogni ritaglio.

- Problema principale: **costo computazionale**.
- Soluzione: implementarla **convoluzionalmente**.

YOLO: griglia $S \times S$; ogni oggetto assegnato a **una cella** (quella del suo centro), che ne predice box e classe.

Anchor box

Per ogni cella, n forme predefinite. Se più oggetti cadono nella stessa cella, ognuno va all'anchor più simile per forma (IoU).



Distribuzione del dataset

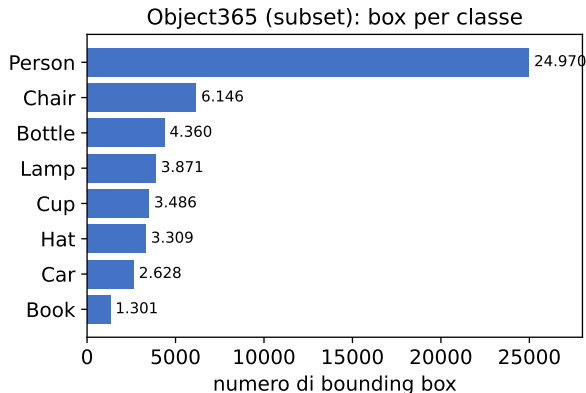
Sottoinsieme fortemente **sbilanciato**:

- **Person** da sola: 49,9% delle box.
- **Book**: appena 2,6%.

⇒ prestazioni attese **inferiori sulle classi rare**.

Gli anchor box sono **scelti a mano** (come indicano le slide), a coprire forme tipiche: piccolo, medio, alto, largo, grande.

Data augmentation (in training): **flip orizzontale** (che ribalta anche le box, $c_x \rightarrow 1 - c_x$) + leggero **color jitter**.



Backbone convoluzionale con downsampling $\times 32$, seguito da una **testa di detection** 1×1 scritta da noi.

Input 3 x 224 x 224

Backbone (downsampling $\times 32$)

-> 512 x 7 x 7 (griglia S)

Head 1x1 -> $A \times (5+C) = 65$ canali

Output (5, 7, 7, 13)

- Rete **completamente convoluzionale**.
- **245** box candidate per immagine ($5 \times 7 \times 7$).

Due backbone a confronto

- **Da zero:** Conv-BN-ReLU + max-pool (15,8M par.).
- **ResNet18 pre-addestrata** su ImageNet, poi fine-tunata (11,2M par.) — *transfer learning*.

In entrambi i casi anchor, target, loss, IoU e NMS restano **implementati a mano**: cambia solo l'estrattore di feature.

La loss YOLO

La rete predice *logit grezzi*, decodificati come (YOLOv2):

$$b_x = \sigma(t_x) + c_x, \quad b_y = \sigma(t_y) + c_y, \quad b_w = a_w e^{t_w}, \quad b_h = a_h e^{t_h}$$

Quattro componenti (normalizzate per numero di oggetti)

Componente	Funzione	Peso	Si applica a
Coordinate	MSE	$\lambda = 5$	anchor responsabili
Objectness	BCE	1	anchor responsabili
No-object	BCE	$\lambda = 0,5$	anchor non responsabili
Classe	Cross-entropy	1	anchor responsabili

Pesi asimmetrici: su 245 box quasi tutte sono vuote $\Rightarrow$ evitano la soluzione degenerare “non c'è mai niente”.

Ogni box: **score** = objectness $\times$ prob. classe.

- 1 Scarta le box sotto soglia (0,25).
- 2 **NMS** per classe.

Non-max suppression

ordina le box per score

finché restano box:

prendi quella con score max $\rightarrow$ predizione

scarta le altre con IoU $>$ soglia

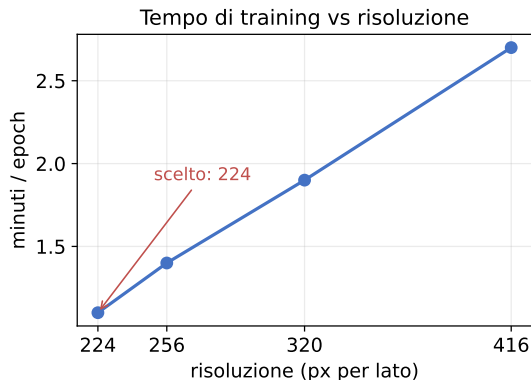
NMS *per classe*: due oggetti di classi diverse possono sovrapporsi (una persona con un cappello); due box sovrapposte della stessa classe no.

Valutazione e ottimizzazione dei tempi

Metrica: precision e recall a $\text{IoU} \geq 0,5$ (l'IoU è la misura del corso), implementata a mano. Niente mAP.

Stato del training

Il training finale **non è ancora stato completato**: nessun valore di precision/recall disponibile. Tutti i componenti sono però implementati e verificati.



Ridurre la risoluzione dimezza i tempi; scelto 224 (~23 min).

- 1 Introduzione: reti neurali
- 2 Object Detection (YOLO)
- 3 Sentiment Analysis (Transformer)**
- 4 Conclusioni

Il problema: classificare frasi

Sentiment analysis

Classificare una sequenza testuale come **positiva** o **negativa**, usando un'architettura **transformer**.

- **Dataset:** SST-2 (frasi brevi etichettate pos/neg, da HuggingFace).
- Corrisponde alla traccia (“frasi positive o negative”); nessun token Kaggle richiesto.
- Valutazione sul *validation set* (872 frasi mai viste).

Il codice supporta comunque un CSV Kaggle via `--dataset csv`.

Dai limiti delle RNN all'attention

Le RNN codificano bene la storia recente, ma faticano su contesti lontani e sono **sequenziali** (lente).

Esempio delle slide

"I grew up in France, ... thus I speak ??" — la parola da predire (*French*) dipende da un termine molto **lontano** nella sequenza.

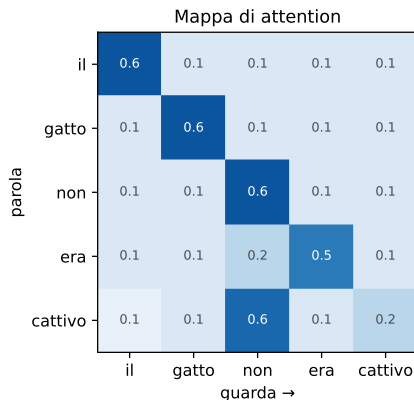
La risposta è il **meccanismo di attention**: per produrre la rappresentazione di una parola, la si mette in relazione con **tutte** le altre e si pesa quali sono rilevanti, indipendentemente dalla distanza.

Self-attention

L'output per ogni parola è una **somma pesata** delle rappresentazioni di tutte le parole; i pesi misurano quanto ognuna è *rilevante*.

Proprietà (dalle slide)

- Calcolo **parallelo** di tutti gli output (non sequenziale).
- *Permutation equivariant*: è più un modello di **insiemi** che di sequenze $\Rightarrow$ serve il **positional encoding** per reintrodurre l'ordine.



cattivo mette attenzione su *non*.

Query, Key, Value e multi-head

Tre ruoli, tre proiezioni lineari apprese

- **Query**: il vettore che viene confrontato con tutti gli altri.
- **Key**: il vettore contro cui la query è confrontata.
- **Value**: ciò che si somma (pesato) per formare l'output.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right) V$$

- Divisione per $\sqrt{d}$: **scaled** attention (stabilità).
- **Multi-head**: più “teste” in parallelo, ciascuna cattura un tipo diverso di relazione tra le parole.

Il blocco Transformer

Un blocco impila due sotto-strati

- 1 **Multi-head self-attention**
- 2 **Rete feed-forward** (applicata a ogni posizione)

Ciascuno con una **connessione residua** (skip) e una **layer normalization**:

$$\text{LayerNorm}(x + \text{Sublayer}(x))$$

Impilando più blocchi si ottiene il Transformer.

Positional encoding

La self-attention è *permutation equivariant*: si **somma** a ogni token un vettore che ne codifica la **posizione**, così il modello conosce l'ordine.

Layer normalization

Come la batch norm, ma normalizza sulle **feature di ogni token** (non sul batch).

Il Transformer e BERT

Transformer

Un modello che usa principalmente la **self-attention** per propagare l'informazione tra le unità (token di testo, ma anche pixel: Vision Transformer).

BERT: transformer **encoder-only**

bidirezionale, pre-addestrato con il **masking** (predire token nascosti). Adatto a *comprendere* una frase, non a generarla.

Il nostro uso

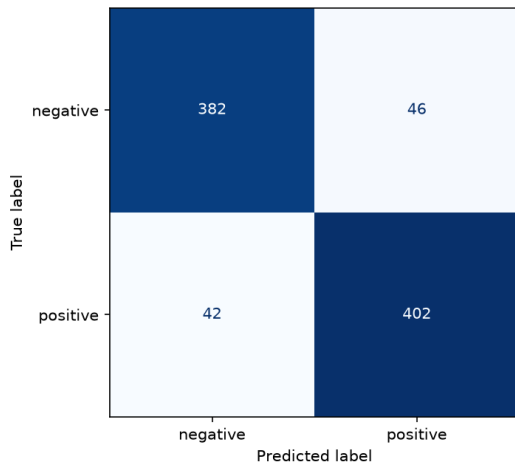
Fine-tuning (transfer learning): si parte da BERT pre-addestrato e si aggiunge una testa di classificazione binaria. Modello: **DistilBERT**, un BERT più leggero.

Risultati

Fine-tuning su MPS (~ 18 min). Metriche sul validation set:

Metrica	Valore
Accuracy	89,9%
F1	0,901
Precision	0,897
Recall	0,905

Errori **bilanciati** (46 vs 42): nessun bias di classe.



Analisi qualitativa: il caso della negazione

Frase costruite ad hoc:

Output del modello

```
[positive 99.9%] I absolutely loved this film
[negative 99.4%] the worst purchase I have ever made
[positive 99.9%] It is not bad at all, quite enjoyable
[negative 69.3%] predictable but the acting saved it
```

- “*not bad*”: contiene *bad* ma è classificata **positiva** al 99,9%. L'attention collega *bad* a *not* — è l'**esempio della slide 11**.
- Frase ambigua $\Rightarrow$ confidenza bassa (69%): il modello *segnala* l'incertezza.

- 1 Introduzione: reti neurali
- 2 Object Detection (YOLO)
- 3 Sentiment Analysis (Transformer)
- 4 Conclusioni**

	Esercizio 1 — Detection	Esercizio 2 — Sentiment
Codice	Completo (869 righe)	Completo (412 righe)
Pipeline verificata	Si', end-to-end	Si', end-to-end
Modello addestrato	No	Si'
Risultati	Non disponibili	Acc. 89,9% — F1 0,901

- **Es. 2 concluso:** risultati solidi, allineati a DistilBERT su SST-2.
- **Es. 1 completo nell'implementazione**, ma manca il training finale (~ 23 min).

Aderenza al programma del corso

Il progetto usa **solo tecniche viste a lezione**: YOLO, anchor, IoU, NMS; CNN; **transfer learning** (backbone ResNet18 pre-addestrato, fine-tuning di BERT); transformer, self-attention; Adam, batch norm; data augmentation.

Rimosso perché non nel corso → sostituito

- anchor k-means → **anchor scelti a mano**
- meccanismo di “ignore” → rimosso
- metrica mAP → **precision/recall a IoU $\geq 0,5$**
- LeakyReLU → **ReLU**; cosine + gradient clipping → rimossi

Mantenuti solo i dettagli **essenziali** (senza cui i modelli non convergono): parametrizzazione sigmoid/exp e pesi λ della loss; encoder BERT pre-addestrato + tokenizer.

Grazie per l'attenzione

Matteo Barbieri — Progetto di Apprendimento Automatico e Profondo
Object Detection & Sentiment Analysis